Labs
Machine Learning Course
Fall 2026

Westlake University
Department of Artificial Intelligence, SOE
Tao Lin
https://github.com/LINs-lab/course_machine_learning

Problem Set Lab 06 (graded), Nov. 12, 2026 (SVM)

Goals. The goal of this exercise is to

- Implement and debug Support Vector Machine (SVM) using SGD and coordinate descent.
- Derive updates for the coordinate descent algorithm for the dual optimization problem for SVM.
- Implement and debug the coordinate descent algorithm.
- Compare it to the primal solution.

Setup, data and sample code. Obtain the folder `labs/lab06_graded` of the course github repository

github.com/LINs-lab/course_machine_learning

We will finally depart from using the height-weight dataset and instead use a toy dataset from scikit-learn. We have provided sample code templates that already contain useful snippets of code required for this exercise.

Additional theory questions. After the main SVM exercises, you will study convexity, softmax cross-entropy and its convexity, and the EM algorithm for a mixture of linear regression models. These are written exercises; no additional code is required.

1 Support Vector Machines using SGD [25 pts]

Until now we have implemented linear and logistic regression to do classification. In this exercise we will use the Support Vector Machine (SVM) for classification. As we have seen in the lecture notes, the original optimization problem for the Support Vector Machine (SVM) is given by

$$\min_{\mathbf{w}\in\mathbb{R}^D} \frac{1}{N}\sum_{n=1}^{N} \ell(y_n \mathbf{X}_n^\top \mathbf{w}) + \frac{\lambda}{2}||\mathbf{w}||^2 \tag{1}$$

where $\ell : \mathbb{R} \to \mathbb{R}$, $\ell(z) := \max\{0, 1-z\}$ is the *hinge loss* function. Here $\lambda > 0$ is the regularization parameter. For any n, $1 \le n \le N$, the vector $\mathbf{X}_n \in \mathbb{R}^D$ is the nth data example, and $y_n \in \{\pm 1\}$ is the corresponding label.

Problem 1 (SGD for SVM [25 pts]):

Implement stochastic gradient descent (SGD) for the original SVM formulation (1). That is in every iteration, pick one data example $n \in [N]$ uniformly at random, and perform an update on $\mathbf{w}$ based on the (sub-)gradient of the nth summand of the objective (1). Then iterate by picking the next n.

1. Fill in the notebook functions `calculate_accuracy(y, X, w)` which computes the accuracy on the training/test dataset for any $\mathbf{w}$ and `calculate_primal_objective(y, X, w, lambda_)` which computes the total primal objective (1).
2. Derive the SGD updates for the original SVM formulation and fill in the notebook function `calculate_stochastic_gradient()` which should return the stochastic gradient of the total cost function (loss plus regularizer) with respect to $\mathbf{w}$. Finally, use `sgd_for_svm_demo()` provided in the template for training.

2 Support Vector Machines using Coordinate Descent [35 pts]

As seen in class, another approach to train SVMs is by considering the dual optimization problem given by

$$\max_{\boldsymbol{\alpha}\in\mathbb{R}^N} \ \frac{1}{N}\boldsymbol{\alpha}^\top\mathbf{1} - \frac{1}{2\lambda N^2}\boldsymbol{\alpha}^\top\mathbf{Y}\mathbf{X}\mathbf{X}^\top\mathbf{Y}\boldsymbol{\alpha} \quad \text{ such that } \quad 0 \le \alpha_n \le 1 \ \forall n \in [N] \tag{2}$$

where $\mathbf{1}$ is the vector of size N filled with ones, $\mathbf{Y} := \mathrm{diag}(\mathbf{y})$, and $\mathbf{X} \in \mathbb{R}^{N\times D}$ again collects all N data examples as its rows, as usual. In this approach we optimize over the dual variables $\boldsymbol{\alpha}$ and map the solutions back to the primal vector $\mathbf{w}$ via the mapping we have seen in class: $\mathbf{w}(\boldsymbol{\alpha}) = \frac{1}{\lambda N}\mathbf{X}^T\mathbf{Y}\boldsymbol{\alpha}$.

Problem 2 (Coordinate Descent for SVM [35 pts]):

In this part we will derive the coordinate descent (or rather ascent in our case) algorithm seen in class to solve the dual (2) of the SVM formulation. That is, at every iteration, we pick a coordinate $n \in [N]$ uniformly at random, and fully optimize the objective (2) **with respect to that coordinate alone**. Hence one step of coordinate ascent corresponds to solving, for a given coordinate $n \in \big[N\big]$ and our current vector $\boldsymbol{\alpha} \in [0,1]^N$, the one dimensional problem:

$$\max_{\gamma\in\mathbb{R}} f(\boldsymbol{\alpha} + \gamma\mathbf{e}_n) \quad \text{ such that } \quad 0 \le \alpha_n + \gamma \le 1 \tag{3}$$

where $f(\boldsymbol{\alpha}) = \frac{1}{N}\boldsymbol{\alpha}^\top\mathbf{1} - \frac{1}{2\lambda N^2}\boldsymbol{\alpha}^\top\mathbf{Y}\mathbf{X}\mathbf{X}^\top\mathbf{Y}\boldsymbol{\alpha}$ and $\mathbf{e}_n = [0,\cdots,1,\cdots,0]^\top$ (all zero vector except at the nth position). We denote by γ^* the value of γ which maximises problem 3. The coordinate update is then $\boldsymbol{\alpha}^{\text{new}} = \boldsymbol{\alpha} + \gamma^*\mathbf{e}_n$.

1. Solve problem 3 and give a closed form solution for the update $\boldsymbol{\alpha}^{\text{new}} = \boldsymbol{\alpha} + \gamma^*\mathbf{e}_n$, this update should only involve $\boldsymbol{\alpha}$, λ, N, $\mathbf{X}_n$, y_n and $\mathbf{w}(\alpha)$ (*Hint:* Notice that $\gamma \mapsto f(\alpha + \gamma\mathbf{e}_n)$ is polynomial and **don't forget the constraints !**). Convince yourself that this update can be computed in $O(D)$ time.
2. Find an efficient way of updating the corresponding $\mathbf{w}(\boldsymbol{\alpha}^{\text{new}})$. This should be computable in $O(D)$ time.
3. Fill in the notebook functions `calculate_coordinate_update()` which should compute the coordinate update for a single desired coordinate and `calculate_dual_objective()` which should return the objective (loss) for the dual problem (2) .
4. Finally train your model using coordinate descent (here ascent) using the given function `coordinate_descent_for_svm_demo()` in the template. Compare to your SGD implementation. Which one is faster? (Compare the training objective values (1) for the $\mathbf{w}$ iterates you obtain from each method). Is the gap going to 0?

Additional Theory Exercises [40 pts]

Convexity [15 pts]

Recall that we say that a function f is *convex* if the domain of f is a convex set and

$$f(\theta\mathbf{X} + (1-\theta)\mathbf{y}) \le \theta f(\mathbf{X}) + (1-\theta)f(\mathbf{y}), \text{ for all } \mathbf{X},\mathbf{y} \text{ in the domain of } f,\ 0 \le \theta \le 1.$$

And *strictly convex* if

$$f(\theta\mathbf{X} + (1-\theta)\mathbf{y}) < \theta f(\mathbf{X}) + (1-\theta)f(\mathbf{y}), \text{ for all } \mathbf{X} \neq \mathbf{y} \text{ in the domain of } f,\ 0 < \theta < 1.$$

Prove the following assertions.

1. The affine function $f(x) = ax + b$ is convex, where a, b and x are scalars.
2. If multiple functions $f_n(\mathbf{X})$ are convex over a fixed domain, then their sum $g(\mathbf{X}) = \sum_n f_n(\mathbf{X})$ is convex over the same domain.

3. Take $f, g : \mathbb{R} \to \mathbb{R}$ to be convex functions and g to be increasing. Then the function $g \circ f$ defined as $(g \circ f)(x) = g(f(x))$ is also convex.

   Note: A function g is increasing if $a \geq b \Leftrightarrow g(a) \geq g(b)$. An example of a convex and increasing function is $\exp(x), x \in \mathbb{R}$.

4. If $f : \mathbb{R} \to \mathbb{R}$ is convex, then $g : \mathbb{R}^D \to \mathbb{R}$, where $g(\mathbf{X}) := f(\mathbf{w}^\top \mathbf{X} + b)$, is also convex. Here, $\mathbf{w}$ is a constant vector in $\mathbb{R}^D$, b is a constant in $\mathbb{R}$ and $\mathbf{X} \in \mathbb{R}^D$.

5. Let $f : \mathbb{R}^D \to \mathbb{R}$ be strictly convex. Let $\mathbf{X}^\star \in \mathbb{R}^D$ be a global minimizer of f. Show that this global minimizer is unique. Hint: Do a proof by contradiction.

Softmax Cross-Entropy and Convexity [15 pts]

Let $N, D \geq 1$ and $K \geq 2$ be integers. Consider classification examples $\{(\mathbf{X}_n, y_n)\}_{n=1}^N$, where $\mathbf{X}_n \in \mathbb{R}^D$ and $y_n \in \{1, \ldots, K\}$. Let $\mathbf{X} \in \mathbb{R}^{N \times D}$ have rows $\mathbf{X}_n^\top$, and write $\mathbf{y} = (y_1, \ldots, y_N)^\top$. For parameters $\mathbf{w}_k \in \mathbb{R}^D$, collected as $\mathbf{W} = (\mathbf{w}_1, \ldots, \mathbf{w}_K)$, define the class scores (*logits*) and probabilities by

$$\eta_{nk} := \mathbf{w}_k^\top \mathbf{X}_n, \qquad p_{nk} := \mathbb{P}[y_n = k \mid \mathbf{X}_n, \mathbf{w}_1, \ldots, \mathbf{w}_K] = \frac{\exp(\eta_{nk})}{\sum_{j=1}^K \exp(\eta_{nj})}.$$

The reference-class constraint $\mathbf{w}_K = \mathbf{0}$ removes the redundancy from adding the same vector to all class parameters; under this constraint, $\mathbf{w}_1, \ldots, \mathbf{w}_{K-1}$ are free. Assume the labels are independent conditional on the inputs. Let $t_{nk} := \mathbf{1}[y_n = k]$, where $\mathbf{1}[A]$ is 1 if statement A holds and 0 otherwise. Define the K-dimensional column vectors

$$\boldsymbol{\eta}_n = (\eta_{n1}, \ldots, \eta_{nK})^\top, \quad \boldsymbol{p}_n = (p_{n1}, \ldots, p_{nK})^\top, \quad \boldsymbol{t}_n = (t_{n1}, \ldots, t_{nK})^\top,$$

and the log-sum-exp function

$$\mathrm{LSE}(\boldsymbol{\eta}_n) := \log \left(\sum_{j=1}^K \exp(\eta_{nj}) \right).$$

All logarithms in these exercises are natural logarithms.

1. Derive the mean negative log-likelihood

$$L(\mathbf{W}) := -\frac{1}{N} \log \mathbb{P}[\mathbf{y} \mid \mathbf{X}, \mathbf{W}].$$

   Here $\ell_n := -\log p_{n,y_n}$ denotes the negative log-likelihood of the observed label for sample n. Express L as the mean cross-entropy between the one-hot targets $\boldsymbol{t}_n$ and the predicted probabilities $\boldsymbol{p}_n$. Also express each sample's loss ℓ_n using $\mathrm{LSE}(\boldsymbol{\eta}_n)$ and the score of its true class.

2. First regard ℓ_n as a function of unrestricted logits $\boldsymbol{\eta}_n \in \mathbb{R}^K$, before imposing the reference-class constraint, and derive $\partial \ell_n / \partial \eta_{nk}$ for every k. Express your answer using p_{nk} and t_{nk}. Then use the chain rule to derive $\nabla_{\mathbf{w}_k} L$ for each free parameter vector $k \in \{1, \ldots, K-1\}$.

3. First allow all K parameter vectors to vary and show that $L(\mathbf{W})$ is jointly convex in $\mathbf{w}_1, \ldots, \mathbf{w}_K$. Then explain why it remains convex in the free parameters when $\mathbf{w}_K = \mathbf{0}$ is imposed.
   Hint: you can use Hölder's inequality:

$$\sum_{k=1}^K |u_k v_k| \leq \left(\sum_{k=1}^K |u_k|^r \right)^{\frac{1}{r}} \left(\sum_{k=1}^K |v_k|^s \right)^{\frac{1}{s}},$$

   for arbitrary $\mathbf{u}, \mathbf{v} \in \mathbb{R}^K$ and exponents $r, s > 1$ satisfying $\frac{1}{r} + \frac{1}{s} = 1$.

Mixture of Linear Regression via EM [10 pts]

Let $N, D, K \geq 1$ be integers. Consider regression examples $\{(\mathbf{X}_n, y_n)\}_{n=1}^N$, where $\mathbf{X}_n \in \mathbb{R}^D$ and $y_n \in \mathbb{R}$. Let $\mathbf{X} \in \mathbb{R}^{N \times D}$ have rows $\mathbf{X}_n^\top$, write $\mathbf{y} = (y_1, \ldots, y_N)^\top$, and define $\tilde{\mathbf{X}}_n := [1, \mathbf{X}_n^\top]^\top \in \mathbb{R}^{D+1}$. Each component has parameters $\mathbf{w}_k \in \mathbb{R}^{D+1}$, collected as $\mathbf{w} = (\mathbf{w}_1, \ldots, \mathbf{w}_K)$. Write $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_K)^\top$ for the mixing weights. Let $r_n \in \{1, \ldots, K\}$ be the latent component for sample n, and write $\boldsymbol{r} = (r_1, \ldots, r_N)^\top$. Conditional on the inputs, assume the pairs (y_n, r_n) are independent across n, with

$$p(r_n = k \mid \boldsymbol{\pi}) = \pi_k, \qquad p(y_n \mid \mathbf{X}_n, r_n = k, \mathbf{w}) = \mathcal{N}(y_n \mid \mathbf{w}_k^\top \tilde{\mathbf{X}}_n, \sigma^2),$$

where $\mathcal{N}(\cdot \mid \mu, \sigma^2)$ denotes a univariate Gaussian density with mean μ and variance σ^2. The variance $\sigma^2 > 0$ is known and shared across components, and $\pi_k \geq 0$, $\sum_{k=1}^K \pi_k = 1$. We will transfer the EM algorithm from Gaussian mixtures to this regression model.

1. Marginalize r_n to obtain $p(y_n \mid \mathbf{X}_n, \mathbf{w}, \boldsymbol{\pi})$. Write the negative log-likelihood $\mathcal{L}(\mathbf{w}, \boldsymbol{\pi})$ for all N observations.

2. Given current parameters $(\mathbf{w}^{\text{old}}, \boldsymbol{\pi}^{\text{old}})$ with $\pi_k^{\text{old}} > 0$ and $\sum_k \pi_k^{\text{old}} = 1$, derive the E-step responsibilities

$$\gamma_{nk} := p(r_n = k \mid \mathbf{X}_n, y_n, \mathbf{w}^{\text{old}}, \boldsymbol{\pi}^{\text{old}})$$

   using Bayes' rule. Explain their interpretation as soft assignments.

3. Keep these responsibilities fixed throughout the M-step. In this M-step, take candidate mixing weights with $\pi_k > 0$ and $\sum_k \pi_k = 1$. Let $\mathbb{E}_{\text{old}}$ denote expectation over $\boldsymbol{r}$ conditional on the observed $(\mathbf{X}, \mathbf{y})$ under the old parameters, and define

$$Q(\mathbf{w}, \boldsymbol{\pi} \mid \mathbf{w}^{\text{old}}, \boldsymbol{\pi}^{\text{old}}) := \mathbb{E}_{\text{old}} \left[\log p(\mathbf{y}, \boldsymbol{r} \mid \mathbf{X}, \mathbf{w}, \boldsymbol{\pi}) \right].$$

   Here $(\mathbf{w}, \boldsymbol{\pi})$ are the candidate parameters being optimized, and superscript new denotes the updated values. Expand Q, ignoring terms independent of these candidate parameters. Show that updating each $\mathbf{w}_k$ is a weighted least-squares problem and write its normal equations. Finally, derive the update for each π_k subject to $\sum_k \pi_k = 1$. You do not need to form a matrix inverse or estimate σ^2.